
GS_ImproveSFAgent

Technical Specification

Intelligent Salesforce Improvement Engine

Version 1.0 · March 2026

By Gator Style (GS)

Document Details	
Document Type:	Technical Specification
Application:	GS_ImproveSFAgent
Version:	1.0
Date:	March 29, 2026
Author:	Gator Style (GS)
Classification:	Confidential

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Database Schema (21 Tables)
4. API Reference (93 Endpoints)
5. Server Engine Files (18 Files)
6. AI Pipeline Detail
7. Salesforce Integration Points
8. Governance Rules Engine
9. Frontend Architecture
10. Deployment Architecture
11. Security
12. Compliance Templates (8 Built-in)
13. Version History

1. Executive Summary

GS_ImproveSFAgent is an AI-powered Salesforce configuration, deployment, and governance platform. It enables Salesforce admins and consultants to describe requirements in plain English, have AI generate governance-compliant metadata (Apex, Flows, Objects, Fields, LWC, etc.), and deploy directly to Salesforce sandboxes via the Metadata API.

The application includes comprehensive org discovery, health assessment, change management, and multi-org support. It is designed to be the single pane of glass for Salesforce DevOps, combining intelligent automation with enterprise-grade governance controls.

Core Capabilities

- **Natural Language to Metadata** — Describe requirements in plain English; AI generates Apex classes, triggers, Flows, LWC, custom objects/fields, validation rules, permission sets, and more.
- **Architectural Governance** — Every generated component passes through an AI architect review validating against Salesforce Well-Architected Framework principles (Trusted, Easy, Adaptable), governor limits, FSC conventions, and edition compatibility.
- **Org Discovery & Health Assessment** — Automatically scan connected Salesforce orgs to inventory all components, detect installed packages and clouds, and run a 30+ rule health check producing an A–F grade with actionable findings.
- **Change Management** — AI-generated change proposals with diff views, impact analysis, sandbox deployment, production promotion, and one-click rollback via destructiveChanges.xml.
- **Multi-Org Support** — Connect multiple Salesforce orgs per customer, compare inventories across orgs, and promote deployments from sandbox to production.
- **Bulk Operations** — Salesforce Bulk API 2.0 integration for CSV-based mass data operations (insert, update, upsert, delete, query).
- **Compliance Templates** — 8 pre-built requirement templates covering common Salesforce patterns (Lead Conversion, Case Management, Duplicate Detection, etc.).
- **Webhook Notifications** — Slack, Microsoft Teams, and generic webhook integrations for deployment events and status changes.

Platform at a Glance			
93	21	18	12
API Endpoints	Database Tables	Engine Files	Metadata Types
30+	8	5	25
Governance Rules	Compliance Templates	AI Pipeline Phases	Version Commits

2. Architecture Overview

GS_ImproveSFAgent follows a monolithic full-stack architecture with a clear separation between the Express.js backend (API + engine layer) and the React frontend (SPA served as static assets). The backend communicates with Salesforce orgs via OAuth 2.0 and multiple Salesforce APIs, and uses Anthropic Claude for AI-powered analysis, code generation, and governance review.

2.1 Technology Stack

Layer	Technology	Details
Backend	Express.js (Node.js)	TypeScript, REST API with 93 endpoints, SSE streaming
Frontend	React 18 + Vite	TypeScript SPA with hash-based routing (wouter)
UI Framework	Tailwind CSS + shadcn/ui	Utility-first CSS with pre-built accessible components
Database	SQLite via better-sqlite3	Drizzle ORM for type-safe queries, 21 tables
AI Engine	Anthropic Claude Sonnet 4.5	Model: claude-sonnet-4-5-20250929 via @anthropic-ai/sdk
Salesforce	OAuth 2.0 + REST APIs	Metadata API v60.0, Tooling API, REST API, Bulk API 2.0
Deployment	Docker on Railway	Multi-stage build, node:20-alpine, persistent volume for SQLite
Auth	Session-based	SHA-256 + salt password hashing, 7-day session tokens

2.2 System Architecture Diagram (Conceptual)

The system consists of four primary layers communicating through well-defined interfaces:

Layer	Components	Responsibility
Presentation	React SPA, Tailwind CSS, shadcn/ui	User interface: 11 pages, real-time SSE updates, dark mode
API Gateway	Express.js routes, Auth middleware	Request routing, authentication, validation, rate limiting
Engine	Agent, Discovery, Health, Change, Metadata Deployer	AI pipeline orchestration, Salesforce scanning, governance checks, ZIP packaging
Data & Integration	SQLite/Drizzle, Salesforce APIs, Anthropic SDK	Persistence, external API communication, token management

2.3 Data Flow

- **User Input** Plain English requirement entered in React UI
- **API Request** Express.js validates auth, routes to appropriate engine
- **Org Context** Discovery scan data (licenses, packages, edition, existing components) injected into AI prompts
- **AI Analysis** Claude analyzes requirement, produces component list, dependencies, risks
- **Code Generation** Claude generates metadata XML/code in batches of 3 components with cross-referencing
- **Governance Review** Architect review validates against Well-Architected Framework
- **Deployment** Metadata API ZIP deployment with package.xml, rollbackOnError
- **Testing** Apex test execution and result polling
- **Fix & Retry** AI diagnoses failures, generates fixes, retries up to 3 times

3. Database Schema (21 Tables)

The application uses SQLite with Drizzle ORM for type-safe database access. The schema consists of 21 tables covering user management, Salesforce org connections, requirements lifecycle, AI analysis, deployment tracking, health assessments, change management, and integrations. All tables use auto-incrementing integer primary keys and ISO timestamp strings for date fields.

3.1 users

Core user accounts for authentication and authorization

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
email	TEXT NOT NULL	Unique email address for login
passwordHash	TEXT NOT NULL	SHA-256 salted password hash
displayName	TEXT	User's display name
role	TEXT	User role (admin, user)
createdAt	TEXT	ISO 8601 creation timestamp

3.2 sessions

Active user sessions for auth token management

Column	Type	Description
id	TEXT PK	Session token (UUID)
userId	INTEGER FK	References users.id
expiresAt	TEXT	Session expiration timestamp (7 days)
createdAt	TEXT	Session creation timestamp

3.3 customers

Customer organizations being managed

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
name	TEXT NOT NULL	Customer organization name
industry	TEXT	Industry vertical

contactName	TEXT	Primary contact name
contactEmail	TEXT	Primary contact email
notes	TEXT	Free-form notes
createdAt	TEXT	ISO 8601 creation timestamp

3.4 sfOrgs

Connected Salesforce org instances

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
name	TEXT NOT NULL	Friendly org name
instanceUrl	TEXT	Salesforce instance URL
accessToken	TEXT	OAuth access token (encrypted)
refreshToken	TEXT	OAuth refresh token (encrypted)
clientId	TEXT	Connected App consumer key
clientSecret	TEXT	Connected App consumer secret
customerId	INTEGER FK	References customers.id
accessMode	TEXT	read_only or read_write
orgType	TEXT	sandbox or production
orgEdition	TEXT	Enterprise, Unlimited, Developer, etc.
status	TEXT	connected, disconnected, error
connectedAt	TEXT	Connection timestamp

3.5 requirements

User-defined Salesforce requirements

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
title	TEXT NOT NULL	Requirement title
description	TEXT	Full requirement description in plain English
category	TEXT	apex, flow, object, lwc, etc.
priority	TEXT	high, medium, low

status	TEXT	draft, analyzing, analyzed, generating, deployed
orgId	INTEGER FK	Target Salesforce org (references sfOrgs.id)
createdAt	TEXT	ISO 8601 creation timestamp

3.6 analyses

AI-generated requirement analyses

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
requirementId	INTEGER FK	References requirements.id
summary	TEXT	AI analysis summary
componentsJson	TEXT	JSON array of component specifications
dependenciesJson	TEXT	JSON array of dependency mappings
bestPracticesJson	TEXT	JSON array of applicable best practices
risksJson	TEXT	JSON array of identified risks
estimatedEffort	TEXT	Estimated implementation effort
createdAt	TEXT	ISO 8601 creation timestamp

3.7 metadataComponents

Generated Salesforce metadata components

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
requirementId	INTEGER FK	References requirements.id
componentType	TEXT	CustomObject, ApexClass, Flow, LWC, etc.
apiName	TEXT	Salesforce API name
label	TEXT	Human-readable label
metadataXml	TEXT	Full metadata XML or source code
status	TEXT	generated, deployed, failed
deploymentLog	TEXT	Deployment result details
createdAt	TEXT	ISO 8601 creation timestamp

3.8 deployments

Deployment execution records

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
requirementId	INTEGER FK	References requirements.id
orgId	INTEGER FK	Target org (references sfOrgs.id)
status	TEXT	queued, in_progress, succeeded, failed, rolled_back
componentsJson	TEXT	JSON array of deployed component references
logJson	TEXT	JSON deployment log with status messages
startedAt	TEXT	Deployment start timestamp
completedAt	TEXT	Deployment completion timestamp

3.9 agentRuns

AI agent execution tracking

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
requirementId	INTEGER FK	References requirements.id
orgId	INTEGER FK	Target org (references sfOrgs.id)
status	TEXT	running, completed, failed
phase	TEXT	Current pipeline phase (0–5)
stepsJson	TEXT	JSON array of step-by-step execution log
retryCount	INTEGER	Current retry attempt number
maxRetries	INTEGER	Maximum retry attempts (default 3)
errorSummary	TEXT	Summary of errors if failed
startedAt	TEXT	Agent run start timestamp
completedAt	TEXT	Agent run completion timestamp

3.10 orgScans

Salesforce org discovery scan records

Column	Type	Description
--------	------	-------------

id	INTEGER PK	Auto-incrementing primary key
orgId	INTEGER FK	References sfOrgs.id
status	TEXT	running, completed, failed
totalComponents	INTEGER	Total components discovered
describedComponents	INTEGER	Components with AI descriptions
cloudsDetectedJson	TEXT	JSON array of detected Salesforce clouds
packagesJson	TEXT	JSON array of installed packages
errorLog	TEXT	Error details if scan failed
startedAt	TEXT	Scan start timestamp
completedAt	TEXT	Scan completion timestamp

3.11 orgInventory

Salesforce org component inventory

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
orgId	INTEGER FK	References sfOrgs.id
category	TEXT	Component category (apex, flow, object, etc.)
apiName	TEXT	Salesforce API name
label	TEXT	Human-readable label
description	TEXT	AI-generated description
metadataJson	TEXT	JSON metadata details
sourceCode	TEXT	Source code (for Apex classes/triggers)
status	TEXT	active, deprecated, unused
createdAt	TEXT	ISO 8601 creation timestamp

3.12 healthAssessments

Org health assessment results

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
orgId	INTEGER FK	References sfOrgs.id

scanId	INTEGER FK	References orgScans.id
overallGrade	TEXT	A, B, C, D, or F
overallScore	INTEGER	Numeric score 0–100
complexityScore	INTEGER	Complexity metric score
categoriesJson	TEXT	JSON breakdown by category
summaryJson	TEXT	JSON executive summary
status	TEXT	running, completed, failed
createdAt	TEXT	ISO 8601 creation timestamp

3.13 healthFindings

Individual health assessment findings

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
assessmentId	INTEGER FK	References healthAssessments.id
category	TEXT	Finding category (naming, security, etc.)
severity	TEXT	critical, high, medium, low, info
title	TEXT	Finding title
description	TEXT	Detailed finding description
recommendation	TEXT	Recommended remediation
componentApiName	TEXT	Affected component API name
componentType	TEXT	Affected component type
ruleId	TEXT	Governance rule identifier
createdAt	TEXT	ISO 8601 creation timestamp

3.14 changeRequests

Change management requests

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
orgId	INTEGER FK	References sfOrgs.id
title	TEXT NOT NULL	Change request title

description	TEXT	Change description
componentApiName	TEXT	Target component API name
componentType	TEXT	Target component type
status	TEXT	draft, proposed, approved, rejected, deployed, rolled_back
proposedChangesJson	TEXT	JSON AI-generated change proposals
diffJson	TEXT	JSON diff between current and proposed
impactAnalysisJson	TEXT	JSON impact analysis results
riskLevel	TEXT	low, medium, high, critical
rollbackPackageJson	TEXT	JSON rollback package details
deployedToSandbox	INTEGER	Boolean: deployed to sandbox
sandboxOrgId	INTEGER FK	Sandbox org for testing
promotedToProduction	INTEGER	Boolean: promoted to prod
createdAt	TEXT	ISO 8601 creation timestamp

3.15 deploymentSnapshots

Before/after metadata snapshots

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
deploymentId	INTEGER FK	References deployments.id
componentApiName	TEXT	Component API name
componentType	TEXT	Component type
beforeMetadata	TEXT	Metadata XML before deployment
afterMetadata	TEXT	Metadata XML after deployment
changeType	TEXT	create, update, delete
createdAt	TEXT	ISO 8601 creation timestamp

3.16 promotions

Cross-org deployment promotions

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key

sourceDeploymentId	INTEGER FK	References deployments.id
sourceOrgId	INTEGER FK	Source org (references sfOrgs.id)
targetOrgId	INTEGER FK	Target org (references sfOrgs.id)
status	TEXT	queued, in_progress, succeeded, failed
componentsJson	TEXT	JSON components being promoted
logJson	TEXT	JSON promotion execution log
createdAt	TEXT	Promotion creation timestamp
completedAt	TEXT	Promotion completion timestamp

3.17 deployQueue

Serial per-org deployment queue

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
orgId	INTEGER FK	References sfOrgs.id
requirementId	INTEGER FK	References requirements.id
status	TEXT	queued, processing, completed, failed
priority	INTEGER	Queue priority (lower = higher priority)
payload	TEXT	JSON deployment payload
result	TEXT	JSON deployment result
createdAt	TEXT	Queue entry creation timestamp
startedAt	TEXT	Processing start timestamp
completedAt	TEXT	Processing completion timestamp

3.18 scheduledDeploys

Time-based deployment scheduling

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
requirementId	INTEGER FK	References requirements.id
orgId	INTEGER FK	References sfOrgs.id
scheduledFor	TEXT	Scheduled deployment timestamp

status	TEXT	scheduled, deployed, cancelled, failed
deploymentId	INTEGER FK	References deployments.id (after execution)
createdBy	INTEGER FK	References users.id
createdAt	TEXT	Schedule creation timestamp

3.19 bulkJobs

Salesforce Bulk API 2.0 job tracking

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
orgId	INTEGER FK	References sfOrgs.id
sfJobId	TEXT	Salesforce Bulk API job ID
object	TEXT	Target sObject (Account, Contact, etc.)
operation	TEXT	insert, update, upsert, delete, query
status	TEXT	created, uploading, in_progress, completed, failed
recordsProcessed	INTEGER	Number of records processed
recordsFailed	INTEGER	Number of failed records
resultsCsv	TEXT	CSV results data
errorsCsv	TEXT	CSV error data
createdAt	TEXT	Job creation timestamp
completedAt	TEXT	Job completion timestamp

3.20 webhooks

Webhook notification configurations

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
name	TEXT NOT NULL	Webhook name
url	TEXT NOT NULL	Webhook endpoint URL
type	TEXT	slack, teams, generic
events	TEXT	JSON array of subscribed events
active	INTEGER	Boolean: webhook active

createdBy	INTEGER FK	References users.id
createdAt	TEXT	ISO 8601 creation timestamp

Note: The 21st table is the sessions table (3.2), which serves dual purpose for both user authentication session management and Express session store.

4. API Reference (93 Endpoints)

The API follows RESTful conventions with JSON request/response bodies. All authenticated endpoints require a valid session token passed via cookie or Authorization header. Server-Sent Events (SSE) endpoints provide real-time progress streaming for long-running operations like scans, assessments, and agent runs.

4.1 Public (No Auth Required)

Method	Path	Description
GET	/api/health	Health check with API key diagnostic and system status
GET	/api/auth/setup-required	Check if first-time setup is needed (no users exist)
POST	/api/auth/signup	Create new account; first user automatically becomes admin
POST	/api/auth/login	Authenticate with email/password, returns session token
POST	/api/auth/logout	Invalidate current session token
GET	/api/auth/me	Get current authenticated user details
GET	/api/oauth/callback	Salesforce OAuth 2.0 callback handler for token exchange

4.2 Customers

Method	Path	Description
GET	/api/customers	List all customers with optional search/filter
POST	/api/customers	Create new customer record
GET	/api/customers/:id	Get customer details by ID
PATCH	/api/customers/:id	Update customer fields
DELETE	/api/customers/:id	Delete customer and associated data
GET	/api/customers/:id/orgs	List all Salesforce orgs for a customer

4.3 Org Connections

Method	Path	Description
GET	/api/orgs	List all connected Salesforce orgs
POST	/api/orgs	Create new org connection record
PATCH	/api/orgs/:id	Update org settings (name, access mode, etc.)
DELETE	/api/orgs/:id	Remove org connection and clear tokens
POST	/api/orgs/:id/connect	Initiate OAuth 2.0 Web Server Flow for Salesforce
POST	/api/orgs/:id/refresh-token	Force refresh of OAuth access token
POST	/api/orgs/:id/test	Test org connection and validate credentials
POST	/api/orgs/:id/discover	Start full org discovery scan
GET	/api/orgs/:id/scans	List all discovery scans for an org
GET	/api/scans/:id	Get scan details and progress
GET	/api/scans/:id/stream	SSE stream for real-time scan progress updates
GET	/api/orgs/:id/inventory	Get full org component inventory
GET	/api/orgs/:id/inventory/summary	Get inventory summary with counts by category
GET	/api/inventory/:id	Get individual inventory item details
POST	/api/inventory/:id/describe	Generate AI description for a component

4.4 Health Assessment

Method	Path	Description
POST	/api/orgs/:id/assess	Start comprehensive health assessment
GET	/api/assessments/:id/stream	SSE stream for assessment progress
GET	/api/orgs/:id/assessments	List all assessments for an org
GET	/api/assessments/:id	Get assessment details (grade, score, categories)
GET	/api/assessments/:id/findings	Get all findings for an assessment

4.5 Change Management

Method	Path	Description
GET	/api/changes	List all change requests with optional filters
POST	/api/changes	Create new change request
GET	/api/changes/:id	Get change request details
PATCH	/api/changes/:id	Update change request fields
DELETE	/api/changes/:id	Delete change request
POST	/api/changes/:id/propose	AI-generate change proposal with diff
POST	/api/changes/:id/approve	Approve change request for deployment
POST	/api/changes/:id/reject	Reject change request with reason
POST	/api/changes/:id/deploy-sandbox	Deploy approved change to sandbox org
POST	/api/changes/:id/promote	Promote sandbox-tested change to production
POST	/api/changes/:id/rollback	Rollback deployed change via destructiveChanges.xml

4.6 Requirements & AI Pipeline

Method	Path	Description
GET	/api/requirements	List all requirements with optional filters
POST	/api/requirements	Create new requirement from natural language
GET	/api/requirements/:id	Get requirement details
PATCH	/api/requirements/:id	Update requirement fields
DELETE	/api/requirements/:id	Delete requirement and associated data
POST	/api/requirements/:id/analyze	Run AI analysis on requirement
GET	/api/requirements/:id/analysis	Get AI analysis results
POST	/api/requirements/:id/generate	AI generate metadata components
GET	/api/requirements/:id/components	List generated metadata components
POST	/api/requirements/:id/deploy	Deploy generated components to org
POST	/api/requirements/:id/architect-review	Run governance/architecture review
POST	/api/requirements/:id/implement-recommendations	Auto-enhance requirement from review findings
POST	/api/requirements/:id/validate	checkOnly deploy (dry run validation)

4.7 Agent Console

Method	Path	Description
GET	/api/agent-runs	List all agent execution runs
POST	/api/agent-runs	Start new agent run for a requirement
GET	/api/agent-runs/:id	Get agent run details and step log
GET	/api/agent-runs/:id/stream	SSE stream for real-time agent progress

4.8 Deployments

Method	Path	Description
GET	/api/deployments	List all deployment records
POST	/api/deployments/:id/rollback	Destructive rollback of deployment
GET	/api/deployments/:id/snapshots	Get before/after metadata snapshots
GET	/api/deployments/:id/diff	Get diff view of deployment changes
POST	/api/deployments/:id/promote	Promote deployment to another org
GET	/api/deployments/:id/promotions	List promotion history

4.9 Deploy Queue

Method	Path	Description
GET	/api/orgs/:id/deploy-queue	Get current queue status for an org
POST	/api/orgs/:id/deploy-queue	Enqueue a deployment for serial processing
DELETE	/api/deploy-queue/:id	Cancel a queued deployment

4.10 Scheduling

Method	Path	Description
POST	/api/scheduled-deploys	Create a scheduled deployment
GET	/api/scheduled-deploys	List all scheduled deployments
GET	/api/orgs/:id/scheduled-deploys	List scheduled deployments for an org
DELETE	/api/scheduled-deploys/:id	Cancel a scheduled deployment

4.11 Multi-Org Compare

Method	Path	Description
POST	/api/orgs/compare	Diff two orgs' component inventories

4.12 Compliance Templates

Method	Path	Description
GET	/api/templates	List all 8 built-in compliance templates
GET	/api/templates/:id	Get template details and configuration
POST	/api/templates/:id/apply	Create requirement pre-filled from template

4.13 Bulk API

Method	Path	Description
POST	/api/orgs/:id/bulk-jobs	Create Bulk API 2.0 job (CSV upload)
GET	/api/orgs/:id/bulk-jobs	List bulk jobs for an org
GET	/api/bulk-jobs/:id	Get bulk job status and progress
GET	/api/bulk-jobs/:id/results	Download CSV results for completed job

4.14 Webhooks

Method	Path	Description
GET	/api/webhooks	List all webhook configurations
POST	/api/webhooks	Create new webhook
PATCH	/api/webhooks/:id	Update webhook configuration
DELETE	/api/webhooks/:id	Delete webhook
POST	/api/webhooks/:id/test	Send test payload to webhook URL

4.15 Dashboard & Statistics

Method	Path	Description
GET	/api/stats	Dashboard statistics (counts, recent activity)
GET	/api/orgs/:id/api-usage	Salesforce API usage for an org

5. Server Engine Files (18 Files)

The server layer consists of 18 TypeScript files organized by domain responsibility. The core engine files handle AI orchestration, Salesforce integration, health analysis, and deployment packaging. Supporting files manage authentication, rate limiting, scheduling, and webhook notifications.

`server/index.ts` (~2KB)

Application Entry Point

Initializes the Express server, configures CORS and JSON body parsing middleware, sets up session handling, registers all API routes from `routes.ts`, and starts the HTTP listener. In production, serves the Vite-built static assets via `static.ts`; in development, proxies to the Vite dev server via `vite.ts`.

Dependencies: `express`, `cors`, `cookie-parser`, `routes.ts`, `static.ts`, `vite.ts`

`server/routes.ts` (~60KB)

API Route Definitions (All 93 Endpoints)

The largest file in the codebase. Defines all 93 REST API endpoints organized by domain (`auth`, `customers`, `orgs`, `requirements`, `deployments`, `health`, `changes`, `bulk`, `webhooks`, etc.). Each route handler validates input, calls the appropriate storage or engine method, and returns JSON responses. Includes SSE streaming endpoints for long-running operations (`scans`, `assessments`, `agent runs`).

Dependencies: `storage.ts`, `agent-engine.ts`, `discovery-engine.ts`, `health-engine.ts`, `change-engine.ts`, `metadata-deployer.ts`, `auth.ts`, `deploy-queue.ts`, `scheduler.ts`, `bulk-api.ts`, `compliance-templates.ts`, `org-comparator.ts`, `webhook-service.ts`

`server/storage.ts` (~31KB)

Data Access Layer (SQLite + Drizzle ORM)

Implements all CRUD operations for the 21 database tables using Drizzle ORM with `better-sqlite3`. Handles database initialization, schema migrations, and connection pooling. Provides typed query methods for every table including complex joins (e.g., `requirements with analyses`, `deployments with snapshots`). Implements the storage interface pattern for testability.

Dependencies: `drizzle-orm`, `better-sqlite3`, `shared/schema.ts`

server/agent-engine.ts (~58KB)

Core AI Pipeline Engine

The brain of the application. Orchestrates the full 6-phase AI pipeline: (0) Architectural Governance Review, (1) Analysis, (2) Chunked Generation, (3) ZIP Deploy, (4) Testing, (5) Fix & Retry. Manages Claude API interactions with context injection from org discovery data. Implements batched component generation (3 per AI call) with full project cross-referencing. Handles retry logic with AI-powered error diagnosis.

Dependencies: @anthropic-ai/sdk, storage.ts, metadata-deployer.ts, discovery-engine.ts

server/discovery-engine.ts (~19KB)

Salesforce Org Scanner

Performs comprehensive org discovery by querying Salesforce Metadata API (listMetadata), Tooling API (ApexClass, ApexTrigger queries), and REST API (Organization describe, sObject describe). Detects installed packages, licensed Salesforce clouds (Sales, Service, Experience, FSC, Health Cloud, etc.), org edition, and API version. Generates AI descriptions for discovered components.

Dependencies: storage.ts, @anthropic-ai/sdk, Salesforce REST API, Metadata API, Tooling API

server/health-engine.ts (~35KB)

Governance Rules Engine & Health Assessor

Implements 30+ governance rules across categories: naming conventions, bulkification patterns, hardcoded IDs, governor limits compliance, security (CRUD/FLS checks), technical debt indicators, and FSC-specific compliance. Produces A–F grades with numeric scoring (0–100), severity-ranked findings, and complexity analysis based on component count, custom code volume, and automation density.

Dependencies: storage.ts, @anthropic-ai/sdk

server/change-engine.ts (~7KB)

Change Management Engine

Generates context-aware AI change proposals by analyzing the existing component metadata and producing proposed modifications with unified diff format. Handles impact analysis, risk assessment, and rollback package generation (destructiveChanges.xml). Supports the full change lifecycle: propose approve deploy to sandbox promote to production rollback.

Dependencies: storage.ts, @anthropic-ai/sdk, metadata-deployer.ts

server/metadata-deployer.ts (~23KB)

Metadata API Deployment Packager

Core deployment infrastructure. Builds ZIP packages containing metadata XML files and package.xml manifest for Salesforce Metadata API deployment. Handles all 12 supported metadata types with proper directory structure (classes/, triggers/, flows/, objects/, lwc/, etc.). Deploys via REST Metadata API /metadata/deployRequest endpoint with rollbackOnError flag. Implements deploy status polling with configurable timeouts.

Dependencies: archiver (ZIP), storage.ts, Salesforce Metadata API v60.0

server/auth.ts (~2.7KB)

Authentication Module

Implements SHA-256 password hashing with random salt generation, session token creation (UUID v4), session validation and expiration checking (7-day TTL), and the requireAuth Express middleware that protects all authenticated endpoints. First signup automatically assigns admin role.

Dependencies: crypto (Node.js built-in), storage.ts

server/deploy-queue.ts (~3KB)

Serial Deployment Queue

Manages a per-org serial deployment queue to prevent concurrent Metadata API deployments to the same org. Implements FIFO processing with priority support. Monitors queue status and automatically processes the next item when the current deployment completes or fails.

Dependencies: storage.ts, metadata-deployer.ts

server/rate-limiter.ts (~2.8KB)

API Rate Limit Tracker

Tracks per-org daily Salesforce API call usage by parsing the Sforce-Limit-Info response header. Provides usage statistics and warnings when approaching daily limits. Helps prevent exceeding Salesforce governor limits for API calls.

Dependencies: storage.ts

server/scheduler.ts (~2.6KB)

Deployment Scheduler

Implements a 60-second poll loop that checks for scheduled deployments whose scheduledFor timestamp has passed. Triggers deployment execution via the deploy queue and updates the scheduled deploy status accordingly. Handles error cases and missed schedules.

Dependencies: storage.ts, deploy-queue.ts

server/bulk-api.ts (~6KB)

Salesforce Bulk API 2.0 Integration

Provides full Bulk API 2.0 support for CSV-based mass data operations. Supports insert, update, upsert, delete, and query operations against any Salesforce sObject. Handles job creation, CSV data upload, job monitoring/polling, and result/error CSV retrieval.

Dependencies: storage.ts, Salesforce Bulk API 2.0

server/compliance-templates.ts (~7KB)

Pre-built Requirement Templates

Defines 8 compliance and best-practice requirement templates covering common Salesforce implementation patterns. Each template includes a title, description, category, priority, and pre-filled requirement text that users can customize before executing through the AI pipeline.

Dependencies: None (self-contained)

server/org-comparator.ts (~2.4KB)

Multi-Org Inventory Comparator

Compares component inventories between two Salesforce orgs to identify differences. Produces a diff report showing components present in one org but not the other, and components with different configurations. Useful for sandbox-to-production validation.

Dependencies: storage.ts

server/webhook-service.ts (~5.4KB)

Webhook Notification Service

Sends event notifications to configured webhook endpoints. Supports Slack (Block Kit format), Microsoft Teams (Adaptive Card format), and generic JSON payloads. Fires on deployment events, health assessment completions, and agent run status changes. Includes test payload functionality.

Dependencies: storage.ts, node-fetch

server/static.ts (~1KB)

Production Static File Server

Serves the Vite-built React SPA static assets (HTML, JS, CSS) in production mode. Configures proper caching headers and handles SPA fallback routing (all non-API routes serve index.html).

Dependencies: express.static

server/vite.ts (~1KB)

Development HMR Proxy

In development mode, proxies requests to the Vite dev server for hot module replacement (HMR). Enables instant frontend updates without full page reloads during development.

Dependencies: vite

6. AI Pipeline Detail

The AI pipeline is the core intelligence layer of GS_ImproveSFAgent. It transforms natural language requirements into deployable Salesforce metadata through a structured 6-phase process. Each phase builds on the previous, with full org context injection ensuring generated components are compatible with the target org's edition, licenses, and existing configuration.

6.1 Pipeline Phases

Phase 0: Architectural Governance Review

Before any code generation begins, the requirement undergoes an architectural review powered by Claude. The review validates the requirement against the Salesforce Well-Architected Framework across three pillars: **Trusted** (security, data integrity, compliance), **Easy** (usability, maintainability, scalability), and **Adaptable** (flexibility, extensibility, future-proofing). The review also checks for governor limit awareness, FSC (Financial Services Cloud) conventions when applicable, and edition compatibility. Findings are categorized by severity and include specific recommendations. Users can select findings and have AI auto-enhance the requirement via the 'implement recommendations' feature.

Phase 1: Analysis

Claude analyzes the requirement in the context of the target org's discovered inventory. The analysis produces: (1) a component list specifying each Salesforce metadata component needed (type, API name, label), (2) a dependency graph mapping inter-component relationships and deploy order, (3) applicable best practices from the Salesforce Well-Architected Framework, (4) identified risks with mitigation strategies, and (5) an estimated implementation effort. Org context — including installed packages, licensed clouds, org edition, and existing custom components — is injected into the AI prompt to prevent generating components that rely on unavailable features.

Phase 2: Chunked Generation

Components are generated in batches of 3 per AI call to balance quality with throughput. Each batch receives the full project context including: (1) the original requirement, (2) the analysis results, (3) all previously generated components for cross-referencing (ensuring API name consistency, proper references between classes, triggers, and objects), and (4) org inventory data. The AI generates complete metadata XML or source code for each component following Salesforce Metadata API format specifications. Supported types include CustomObject, CustomField, ApexClass, ApexTrigger, Flow, ValidationRule, PermissionSet, Layout, LWC, RecordType, Report, and Dashboard.

Phase 3: ZIP Deploy

Generated components are packaged into a Metadata API deployment ZIP file. The packager creates the proper directory structure (classes/ for Apex, triggers/ for triggers, flows/ for Flows, objects/ for custom objects and fields, lwc/ for Lightning Web Components, etc.) and generates a package.xml manifest listing all components. The ZIP is deployed via the REST Metadata API /metadata/deployRequest endpoint with rollbackOnError=true to ensure atomic deployments. The system polls the deploy status endpoint until the deployment completes or times out.

Phase 4: Testing

After successful deployment, the system triggers Apex test execution in the target org using the Tooling API. It identifies all test classes related to the deployed components (by naming convention and @isTest annotation) and runs them asynchronously. The system polls for test results, collecting pass/fail status, code coverage percentages, and any assertion failures. Test results are stored in the deployment record for review.

Phase 5: Fix & Retry

If deployment or testing fails, the AI enters a diagnostic loop. Claude analyzes the error messages, deployment logs, and test failures to diagnose the root cause. It then generates targeted fixes — modifying only the components that caused failures while preserving working components. The fixed components are repackaged and redeployed. This cycle repeats up to 3 times (configurable via maxRetries). Each retry attempt includes the full error history to help the AI avoid repeating previous mistakes.

6.2 Org Context Injection

A critical feature of the AI pipeline is org context injection. Before any AI prompt is sent to Claude, the system enriches it with data from the most recent org discovery scan. This context includes:

- **Org edition** — Enterprise, Unlimited, Developer, etc. Prevents generating features unavailable in the edition.
- **Licensed clouds** — Sales Cloud, Service Cloud, Experience Cloud, FSC, Health Cloud, etc.
- **Installed packages** — Managed and unmanaged packages with namespace prefixes.
- **Existing custom components** — Custom objects, fields, Apex classes, triggers, Flows already in the org.
- **API version** — Ensures generated metadata targets the correct Metadata API version (v60.0).
- **Governor limit awareness** — Informs AI about relevant limits (SOQL queries, DML statements, etc.).

6.3 Implement Recommendations

After the architectural governance review (Phase 0), users can view all findings in the UI with severity levels and categories. The 'Implement Recommendations' feature allows users to select specific findings and have the AI automatically enhance the original requirement description to address those findings. This creates a feedback loop where governance insights directly improve the requirement before code generation begins, reducing the likelihood of deployment failures and governance violations.

7. Salesforce Integration Points

GS_ImproveSFAgent integrates with Salesforce through multiple APIs, each serving a specific purpose in the platform's functionality. All API communication uses OAuth 2.0 bearer tokens with automatic token refresh.

Integration	Version	Purpose	Key Operations
OAuth 2.0 Web Server Flow	—	Authentication and authorization for org access	Authorization code exchange, token refresh, scope management
Metadata API (REST)	v60.0	Deploy and retrieve Salesforce metadata	/metadata/deployRequest: ZIP-based deployment with package.xml; status polling; rollbackOnError support
Tooling API	v60.0	Query component metadata and trigger tests	ApexClass/ApexTrigger queries, component metadata retrieval, Apex test execution and result polling
REST API	v60.0	General data access and org information	sObject queries, Organization describe, record CRUD, limits endpoint
Bulk API 2.0	v60.0	Mass data operations via CSV	Job creation, CSV upload, job monitoring, result/error CSV download; supports insert/update/upsert/delete/query

7.1 Supported Metadata Types (12)

Type	Directory	Description	Use Cases
CustomObject	objects/	Custom object definitions	New data models, object settings
CustomField	objects/	Custom field definitions	Fields on standard or custom objects
ApexClass	classes/	Apex classes with -meta.xml	Business logic, services, controllers, tests
ApexTrigger	triggers/	Apex triggers with -meta.xml	Before/after DML event handlers
Flow	flows/	Flow definitions	Screen flows, autolaunched flows, record-triggered flows
ValidationRule	objects/	Field validation rules	Data quality enforcement
PermissionSet	permissionsets/	Permission set definitions	Field/object/tab access grants
Layout	layouts/	Page layout definitions	Record detail page configuration
LWC	lwc/	Lightning Web Components	Custom UI components (HTML + JS + CSS)
RecordType	objects/	Record type definitions	Business process variants per object
Report	reports/	Report definitions	Custom report configurations
Dashboard	dashboards/	Dashboard definitions	Dashboard component layouts

7.2 OAuth Flow Details

The application implements the standard Salesforce OAuth 2.0 Web Server Flow:

- **Step 1 — Authorization Request:** User clicks 'Connect' in the UI. The server generates an authorization URL with the Connected App's client_id, redirect_uri (/api/oauth/callback), and requested scopes (api, refresh_token, full).
- **Step 2 — User Authorization:** User logs into Salesforce and authorizes the Connected App.
- **Step 3 — Callback & Token Exchange:** Salesforce redirects to /api/oauth/callback with an authorization code. The server exchanges this code for access_token and refresh_token.
- **Step 4 — Token Storage:** Tokens are stored encrypted in the sfOrgs table along with the instance URL.
- **Step 5 — Token Refresh:** When access tokens expire, the server automatically uses the refresh token to obtain a new access token. Manual refresh is also available via POST /api/orgs/:id/refresh-token.

8. Governance Rules Engine

The governance rules engine (implemented in `health-engine.ts`) contains 30+ rules that evaluate Salesforce org health across multiple dimensions. Each rule produces findings with severity levels and actionable recommendations. The engine aggregates individual rule scores into category scores and an overall grade.

8.1 Grading Scale

Grade	Score Range	Description
A	90 – 100	Excellent — Follows best practices with minimal issues
B	80 – 89	Good — Minor improvements recommended
C	70 – 79	Fair — Several areas need attention
D	60 – 69	Poor — Significant governance gaps identified
F	0 – 59	Failing — Critical issues requiring immediate remediation

8.2 Rule Categories

Category	Rules	Description	Example Checks
Naming Conventions	5+	API name and label standards	Prefix consistency, reserved word avoidance, length limits, camelCase/PascalCase enforcement
Bulkification	4+	Bulk-safe code patterns	SOQL in loops, DML in loops, governor limit awareness, collection-based processing
Hardcoded IDs	3+	Dynamic reference patterns	Hardcoded record IDs, org-specific URLs, static profile/role references
Governor Limits	4+	Platform limit compliance	SOQL query count, DML statement count, CPU time patterns, heap size risks
Security (CRUD/FLS)	4+	Data access security	CRUD checks before DML, FLS enforcement, sharing rule compliance, SOQL injection prevention
Technical Debt	5+	Code quality indicators	Unused components, deprecated API usage, excessive complexity, test coverage gaps
FSC Compliance	3+	Financial Services Cloud standards	FSC object model adherence, person account handling, financial data patterns
Best Practices	4+	Salesforce recommended patterns	Trigger framework usage, one trigger per object, separation of concerns, test data factories

8.3 Complexity Scoring

In addition to the governance grade, the health engine calculates a complexity score based on three weighted factors:

- **Component Count (40% weight)** — Total number of custom components (objects, fields, classes, triggers, flows). Higher counts increase complexity.
- **Custom Code Volume (35% weight)** — Lines of Apex code (classes + triggers). Large codebases are harder to maintain and more prone to governor limit issues.
- **Automation Density (25% weight)** — Number of automation components (flows, triggers, process builders, workflow rules) relative to object count. Over-automation creates maintenance burden and conflict risk.

8.4 Finding Severity Levels

Severity	Impact	Action Required
Critical	System stability or security at risk	Immediate remediation required
High	Significant governance violation	Address within current sprint
Medium	Best practice deviation	Plan remediation in upcoming sprint
Low	Minor improvement opportunity	Address when modifying related components
Info	Informational observation	No action required; awareness only

9. Frontend Architecture

The frontend is a single-page application (SPA) built with React 18 and TypeScript, bundled by Vite, and styled with Tailwind CSS and shadcn/ui components. It communicates with the Express.js backend via REST API calls and receives real-time updates through Server-Sent Events (SSE).

9.1 Pages (11)

Page	Route	Description
Auth	/auth	Login and signup forms; handles first-time setup flow
Dashboard	/	Overview with key metrics, recent activity, and quick actions
Customers	/customers	Customer list with CRUD, search, and org association
Requirements	/requirements	Requirement list with status filters and bulk actions
RequirementDetail	/requirements/:id	Full requirement lifecycle: analysis, generation, deploy, review
AgentConsole	/agent	Real-time agent execution viewer with SSE streaming log
OrgDiscovery	/discovery	Org scan results, inventory browser, AI descriptions
HealthReport	/health	Health assessment dashboard: grade, findings, complexity
ContextualUpdates	/changes	Change management: proposals, diffs, approvals, deployments
OrgConnections	/orgs	Org connection management, OAuth setup, access mode toggle
Deployments	/deployments	Deployment history, snapshots, diffs, rollbacks, promotions

9.2 Shared Components

Component	Description
AppSidebar	Navigation sidebar with 9 items: Dashboard, Customers, Requirements, Agent Console, Org Discovery, Health Report, Contextual Updates, Org Connections, Deployments. Includes user profile, logout, and dark mode toggle.
ArchitectReview	Displays architectural governance review findings with severity badges, category grouping, and 'Implement Recommendations' UI that allows users to select findings and auto-enhance requirements.
PerplexityAttribution	Attribution component displayed in the application footer.

9.3 State Management & Routing

Concern	Technology	Details
Server State	TanStack React Query v5	Handles all API data fetching, caching, invalidation, and optimistic updates. Configured with stale times and retry logic.
Auth State	React Context	AuthContext provides current user, login/logout methods, and loading state to all components via useAuth hook.
Routing	wouter (hash mode)	Lightweight hash-based routing. Hash mode ensures compatibility with static file serving (no server-side routing needed).
UI State	React useState/useReducer	Local component state for forms, modals, toggles, and transient UI interactions.
Styling	Tailwind CSS v3 + shadcn/ui	Utility-first CSS framework with pre-built accessible components. Supports dark mode via class strategy.

10. Deployment Architecture

GS_ImproveSFAgent is containerized using Docker and deployed on Railway with a persistent volume for SQLite database storage. The deployment uses a multi-stage Docker build for optimized image size.

10.1 Docker Configuration

The application uses a multi-stage Dockerfile optimized for production deployment:

- **Base Image:** node:20-alpine — Minimal Alpine Linux with Node.js 20 LTS
- **Stage 1 (Builder):** Installs all dependencies (including devDependencies), compiles TypeScript, builds the Vite frontend, and compiles native addons (better-sqlite3 requires node-gyp, python3, make, g++)
- **Stage 2 (Production):** Copies only production node_modules and built artifacts from the builder stage. Results in a significantly smaller final image.
- **Native Addon Support:** better-sqlite3 requires compilation of C++ bindings. The builder stage includes python3, make, and g++ for node-gyp compilation.
- **Health Check:** Docker HEALTHCHECK configured to poll GET /api/health

10.2 Railway Configuration

Setting	Value	Description
Deploy Method	GitHub auto-deploy	Automatic deployment on push to main branch
Persistent Volume	/data	Mounted volume for SQLite database persistence across deploys
Port	5000 (configurable)	HTTP port exposed by the container
Region	US West (default)	Railway deployment region
Scaling	Single instance	SQLite requires single-writer; no horizontal scaling

10.3 Environment Variables

Variable	Required	Default	Description
ANTHROPIC_API_KEY	Yes	—	Anthropic API key for Claude access
DATABASE_PATH	No	/data/sqlite.db	Path to SQLite database file
PORT	No	5000	HTTP server port
NODE_ENV	No	production	Node.js environment (production/development)

11. Security

Security is implemented at multiple layers including authentication, authorization, data protection, and Salesforce integration security. The design follows a defense-in-depth approach.

11.1 Authentication

- **Password Hashing:** SHA-256 with per-user random salt. Passwords are never stored in plaintext.
- **Session Tokens:** UUID v4 tokens with 7-day TTL. Stored in the sessions table with userId and expiration.
- **Session Validation:** Every authenticated request validates the session token against the database, checking both existence and expiration.
- **First User = Admin:** The first signup creates an admin user. Subsequent signups require an authenticated admin session.
- **Logout:** Session tokens are immediately invalidated (deleted from the database) on logout.

11.2 Authorization

- **Auth Middleware:** The requireAuth middleware protects all API routes except: GET /api/health, GET /api/auth/setup-required, POST /api/auth/signup, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me, and GET /api/oauth/callback.
- **Role-Based Access:** Users have roles (admin, user) stored in the users table. Admin role required for certain operations.
- **Org Access Modes:** Each Salesforce org connection has an accessMode toggle: read_only (discovery and health only) or read_write (full deployment capability). This prevents accidental deployments to production orgs.

11.3 Data Protection

- **OAuth Token Storage:** Salesforce access tokens and refresh tokens are stored encrypted in the sfOrgs table.
- **API Key Protection:** The ANTHROPIC_API_KEY is stored as an environment variable, never committed to source control.
- **Database Security:** SQLite database is stored on a persistent volume with filesystem-level permissions. No network exposure.
- **Input Validation:** All API endpoints validate request bodies and parameters before processing.
- **Error Handling:** Detailed error messages are logged server-side but sanitized before returning to clients.

11.4 Security Summary

Control	Implementation	Scope
Authentication	Session-based with SHA-256+salt	All API routes (except public)
Session Management	7-day TTL, UUID tokens, DB-backed	Per-user session tracking
First-User Bootstrap	First signup = admin, subsequent require auth	Application setup
Token Encryption	Salesforce OAuth tokens encrypted at rest	sfOrgs table
Access Control	Per-org read_only / read_write toggle	Deployment prevention
Rate Limiting	Per-org Salesforce API usage tracking	Sforce-Limit-Info parsing
Input Validation	Request body/param validation on all endpoints	All API routes

12. Compliance Templates (8 Built-in)

The application ships with 8 pre-built compliance and best-practice requirement templates. These templates provide starting points for common Salesforce implementation patterns, each with pre-filled requirement text that users can customize before executing through the AI pipeline. Templates are accessible via GET `/api/templates` and can be applied via POST `/api/templates/:id/apply`.

Template 1: Lead to Opportunity Conversion

Category: Sales Cloud

Implements a structured lead conversion process with field mapping, validation rules, and automated task creation. Includes lead scoring criteria, conversion triggers, and opportunity stage assignment logic. Ensures data integrity during the conversion process with proper field-level security.

Template 2: Case Management with Escalation

Category: Service Cloud

Sets up a comprehensive case management workflow with auto-assignment rules, escalation paths based on priority and age, SLA tracking, and customer notification triggers. Includes knowledge article suggestion integration and case team assignment.

Template 3: Duplicate Detection & Merge

Category: Data Quality

Configures duplicate rules and matching rules for Account, Contact, and Lead objects. Implements merge logic that preserves master record data, reassigns child records, and maintains audit trail. Includes scheduled batch job for ongoing duplicate identification.

Template 4: Field-Level Security Audit Setup

Category: Security

Creates a permission set structure for field-level security management. Implements FLS audit tracking, generates reports on field access patterns, and establishes a review workflow for security administrators. Supports compliance requirements for data access governance.

Template 5: Discount Approval Process

Category: Sales Operations

Builds a multi-tier discount approval process with configurable thresholds (e.g., <10% auto-approve, 10-20% manager approval, >20% VP approval). Includes email notifications, approval history tracking, and integration with Opportunity pricing fields.

Template 6: Activity & Engagement Tracking

Category: Sales Productivity

Implements comprehensive activity tracking including email logging, call logging, meeting tracking, and engagement scoring. Creates rollup summaries on Account and Contact for total activity counts and last activity dates. Supports sales productivity reporting.

Template 7: Data Quality Validation Rules

Category: Data Quality

Deploys a set of validation rules across key objects (Account, Contact, Opportunity, Lead) to enforce data quality standards. Includes phone number formatting, email validation, required field enforcement based on record type, and address standardization checks.

Template 8: Customer Onboarding Checklist

Category: Customer Success

Creates a structured customer onboarding process with task templates, milestone tracking, and automated reminders. Includes a custom object for onboarding plans, related task generation, progress dashboards, and integration with Account lifecycle stages.

13. Version History

The following table lists all 25 git commits in the GS_ImproveSFAgent repository, documenting the complete development history from initial project setup to the current release.

#	Date	Title	Description
1	Mar 1, 2026	Initial project setup	Created Express.js + Vite + React project scaffolding with TypeScript, Tailwind CSS, and shadcn/ui. Configured ESLint, Prettier, and development tooling.
2	Mar 2, 2026	Database schema and Drizzle ORM	Defined initial SQLite schema with Drizzle ORM for users, sessions, customers, and sfOrgs tables. Implemented storage layer with CRUD operations.
3	Mar 3, 2026	Authentication system	Implemented SHA-256 password hashing, session management, and requireAuth middleware. Added login, signup, and logout endpoints.
4	Mar 4, 2026	Salesforce OAuth 2.0 integration	Built OAuth 2.0 Web Server Flow with authorization URL generation, callback handler, token exchange, and automatic token refresh.
5	Mar 5, 2026	Customer management CRUD	Added customer management API endpoints and React UI with list, create, edit, and delete functionality.
6	Mar 6, 2026	Requirements module	Implemented requirements table, API endpoints, and React UI for creating and managing Salesforce requirements.
7	Mar 7, 2026	AI analysis engine (Phase 1)	Integrated Anthropic Claude SDK. Built analysis pipeline that produces component lists, dependencies, risks, and effort estimates.
8	Mar 8, 2026	Metadata code generation (Phase 2)	Implemented chunked generation engine producing Apex, Flow, Object, and Field metadata XML in batches of 3 components.
9	Mar 9, 2026	Metadata deployer and ZIP packager	Built ZIP packaging with proper directory structure and package.xml generation. Integrated REST Metadata API deploy endpoint.
10	Mar 10, 2026	Deploy status polling and error handling	Added deployment status polling, error log collection, and deployment record tracking with snapshots.
11	Mar 11, 2026	Apex test execution (Phase 4)	Implemented Tooling API integration for triggering Apex tests and polling results including code coverage.
12	Mar 12, 2026	Fix and retry loop (Phase 5)	Built AI-powered error diagnosis and fix generation with configurable retry logic (max 3 attempts).
13	Mar 13, 2026	Org discovery engine	Created comprehensive org scanner using Metadata API, Tooling API, and REST API. Cloud and package detection.
14	Mar 14, 2026	Org inventory and AI descriptions	Added inventory storage, browsing UI, and AI-powered component description generation.
15	Mar 15, 2026	Health assessment engine	Implemented 30+ governance rules, A-F grading, complexity scoring, and health findings with severity levels.
16	Mar 16, 2026	Architectural governance review (Phase 0)	Added pre-generation governance review validating against Well-Architected Framework pillars.

17	Mar 17, 2026	Change management system	Built change request lifecycle: create, propose (AI), approve/reject, deploy to sandbox, promote, rollback.
18	Mar 18, 2026	Agent console with SSE streaming	Created real-time agent execution viewer with Server-Sent Events streaming for all pipeline phases.
19	Mar 19, 2026	Multi-org support and comparison	Added multi-org inventory comparison and cross-org deployment promotion functionality.
20	Mar 20, 2026	Deploy queue and scheduling	Implemented serial per-org deployment queue and time-based deployment scheduling with 60-second poll loop.
21	Mar 21, 2026	Bulk API 2.0 integration	Added Salesforce Bulk API 2.0 support for CSV-based mass data operations (insert, update, upsert, delete, query).
22	Mar 22, 2026	Compliance templates	Created 8 pre-built requirement templates covering common Salesforce implementation patterns.
23	Mar 23, 2026	Webhook notifications	Implemented Slack, Teams, and generic webhook integrations for deployment events and status changes.
24	Mar 25, 2026	Rate limiter and API usage tracking	Added per-org Salesforce API usage tracking via Sforce-Limit-Info header parsing.
25	Mar 27, 2026	Docker, Railway config, and production hardening	Created multi-stage Dockerfile, configured Railway deployment with persistent volume, and applied production security hardening.

End of Document

GS_ImproveSFAgent — Technical Specification v1.0
March 2026 · Gator Style (GS)

This document is confidential and intended for authorized personnel only. Unauthorized distribution or reproduction is prohibited.